

review

June 11, 2025

```
[3]: import pandas as pd
import matplotlib.pyplot as plt

# convert the file into a csv file
df = pd.read_excel('DeepLearning_Evapotranspiration_Papers.xlsx')
df.to_csv('review.csv', index=False)
```

```
[4]: # read the csv file
df = pd.read_csv('review.csv')

# print the column names
print(df.columns)
```

```
Index(['Publication Type', 'Authors', 'Book Authors', 'Book Editors',
      'Book Group Authors', 'Author Full Names', 'Article Title',
      'Source Title', 'Language', 'Document Type',
      ...,
      'Linear detrending', 'RMSprop', 'SGD', 'Adam', 'short term', 'mid term',
      'long term', 'Local', 'Regional', 'Global.1'],
      dtype='object', length=307)
```

```
[5]: # # remove all the columns before the column 'DNN'
df = df.loc[:, 'DNN':]

# convert the values in the dataframe to boolean
df = df.applymap(lambda x: True if x == 'True' else True if x == True else
↳ False)

# write the dataframe to a new csv file
df.to_csv('review.csv', index=False)
```

/tmp/ipykernel_52222/3824094904.py:5: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.

```
df = df.applymap(lambda x: True if x == 'True' else True if x == True else
False)
```

```
[6]: algorithms = [
      'DNN', 'MLP', 'DeepFM', 'BPNN', 'GRNN', 'DBN', 'DL', 'RNN', 'BiRNN',
↳ 'LSTM', 'BiLSTM',
```

```

    'GRU', 'BiGRU', 'SSR-LSTM', 'ConvLSTM', 'NARX', 'CNN', 'YOLO',
    ↪ 'Transformer', 'ADAQNet',
    'QAxNet', 'GAN', 'AE', 'CVAE', 'ELM', 'K-means-FFA-KELM', 'ANFIS', 'MPS',
    ↪ 'QSDM', 'GMDH',
    'SWAT', 'SL', 'PLR', 'eXML', 'MLNR', 'CBR', 'DT', 'CART', 'BCART',
    ↪ 'M5Tree', 'RF', 'DRF',
    'BT', 'GB', 'XGB', 'DF', 'LightGB', 'AdaBoost', 'LR', 'Lasso', 'PL', 'GLM',
    ↪ 'GPR', 'SARIMA',
    'GLM', 'MEMD', 'MARS', 'SAR2ET', 'HBV', 'BMA', 'PT-JPL', 'SR', 'ER', 'SVM',
    ↪ 'SVR', 'RL', 'DRL',
    'MTL', 'STL', 'AutoML'
]

metrics = [
    'RMSE', 'MSE', 'RMSD', 'SEP', 'ubRMSE', 'SDR', 'WSEE', 'E_l,m',
    'MAE', 'WAPE', 'MAD', 'MADE', 'AARE', 'MAPE', 'MdAPE', 'BME',
    'Bias', 'MBE', 'RB', 'RRMSE', 'NRMSE', 'RRSE', 'RAE', 'R',
    'R^2', 'VAF', 'PCC', 'KGE', 'NSE', 'NNSE', 'IOA',
    'Accuracy', 'Precision', 'F1 Score', 'AUC', 'IOU'
]

dataSources = [
    'MODIS', 'Sentinel-1', 'Sentinel-2', 'AVHRR', 'NOAA CDR AVHRR NDVI',
    ↪ 'ASTER', 'SMOS', 'AMSR2', 'OCO-2', 'GRACE',
    'HyTES', 'ECOSTRESS', 'COMS', 'SST', 'GLASS', 'HLS', 'SRTM', 'Copernicus',
    ↪ 'ERA 5', 'MERRA2', 'NLDAS', 'GLDAS',
    'FLDAS', 'NCA-LDAS', 'WFDEI', 'ACCESS-ESM1,5', 'CMIP6', 'CHIRPS', 'SPEI',
    ↪ 'GIOVANNI', 'POWER', 'GLEAM', 'LAADS',
    'EEFLUX', 'NESDC', 'AggieAir sUAS', 'ASDC', 'CSIF', 'PlantVillage',
    ↪ 'FLUXNET', 'SNOTEL', 'ISMN', 'CIMIS',
    'AERONET', 'GRDC', 'USGS', 'INMET', 'HPRCC', 'AZMET', 'FAWN', 'DEOS',
    ↪ 'AWN', 'Weather station', 'Lysimeter',
    'Eddy Covariance', 'CAMELS', 'CANOPEX', 'TAMARACK', 'CRC', 'ACARR', 'LULC',
    ↪ 'DEM', 'HydroSHEDS', 'NALCMS', 'ASCE'
]

locations = [
    'Iran', 'Iraq', 'Egypt', 'Canada', 'Australia', 'U.S.', 'China',
    ↪ 'Colombia', 'India', 'South Korea',
    'Pakistan', 'Bangladesh', 'Israel', 'Vietnam', 'Laos', 'Cambodia',
    ↪ 'Indonesia', 'Malaysia', 'Nepal', 'Slovakia',
    'Sudan', 'Nigeria', 'Ethiopia', 'Algeria', 'Italy', 'Germany', 'Turkey',
    ↪ 'Morocco', 'Peru', 'Ecuador', 'Mexico',
    'Chile', 'Brazil', 'South Africa', 'Portugal', 'Sweden', 'Global', 'Aegean',
    ↪ 'areas', 'Balkans areas', 'Africa',

```

```

    'Amazon', 'Mediterranean areas', 'Oceania areas', 'Savannahs',
    ↪ 'Grasslands', 'Wetlands', 'Croplands', 'Forests'
]

asianLocations = [
    'Iran', 'Iraq', 'China', 'India', 'South Korea', 'Pakistan', 'Bangladesh',
    ↪ 'Israel',
    'Vietnam', 'Laos', 'Cambodia', 'Indonesia', 'Malaysia', 'Nepal', 'Turkey'
]

africanLocations = [
    'Egypt', 'Sudan', 'Nigeria', 'Ethiopia', 'Algeria', 'Morocco', 'South
    ↪ Africa', 'Savannahs', 'Africa'
]

americanLocations = [
    'Canada', 'Colombia', 'Peru', 'Ecuador', 'Mexico', 'Chile', 'Brazil',
    ↪ 'Amazon', 'U.S.'
]

europeanLocations = [
    'Italy', 'Germany', 'Slovakia', 'Portugal', 'Sweden', 'Aegean areas',
    'Balkans areas', 'Mediterranean areas'
]

oceanianLocations = [
    'Australia', 'Oceania areas'
]

generalLocations = [
    'Grasslands', 'Wetlands', 'Croplands', 'Forests', 'Global'
]

variables = ['Meterological variables', 'Soil variables']

optimizations = [
    'GWO', 'HHO', 'PSO', 'SAO', 'WSO', 'FCM', 'K-means', 'PCA', 'ICA', 'SHAP',
    ↪ 'PACF',
    'Transfer Learning', 'Low-pass', 'High-pass', 'Normalization',
    ↪ 'Interpolation', 'Linear detrending',
    'RMSprop', 'SGD', 'Adam'
]

studyTerms = [
    'short term', 'mid term', 'long term'
]

```

```
coverage = [
    'Local', 'Regional', 'Global'
]
```

```
[17]: def plot_histogram(df, metrics_of_interest, focused_classes):
    mask = df[metrics_of_interest].any(axis=1)
    df_metrics = df[mask]
    counts = df_metrics[focused_classes].sum()
    # counts = counts / counts.sum() # Normalize to sum to 1
    asianLocationsCounts = counts[asianLocations].sum()
    europeanLocationsCounts = counts[europeanLocations].sum()
    africanLocationsCounts = counts[africanLocations].sum()
    americanLocationsCounts = counts[americanLocations].sum()
    oceanianLocationsCounts = counts[oceanianLocations].sum()
    generalLocationsCounts = counts[generalLocations].sum()
    # put everything into one dataframe
    combined_counts = pd.DataFrame({
        'Asia': asianLocationsCounts,
        'Europe': europeanLocationsCounts,
        'Africa': africanLocationsCounts,
        'America': americanLocationsCounts,
        'Oceania': oceanianLocationsCounts,
        'General': generalLocationsCounts
    }, index=[0])
    combined_counts = combined_counts.T
    combined_counts.columns = ['Occurrences']
    combined_counts.index.name = 'Locations'
    plt.figure(figsize=(12, 5))
    combined_counts.plot(kind='bar')
    plt.title(f'Locations of the papers')
    plt.xlabel('Locations')
    plt.ylabel('Occurrences')
    plt.xticks(rotation=90)
    plt.tight_layout()
    plt.savefig('locations_histogram.png', dpi=300)
    plt.show()
```

```
[18]: plot_histogram(df, df.columns, locations)
```

<Figure size 1200x500 with 0 Axes>

